Generative AI-Assisted Malware Disassembly and Remediation

How LLMs accelerate reverse engineering and patching (and where they still fail)

Lois Briones · David Esperante · Héctor Aldao

2026-11-16

MUniCS - Malware Analysis

Agenda

1. Context & Motivation
2. The Toolchain
3. Manual vs. AI-Augmented Workflow
4. Evidence: Speed, Accuracy & Hallucinations
5. Live Demo
6. Takeaways

Context & Motivation

1. **Context & Motivation**
2. The Toolchain
3. Manual vs. AI-Augmented Workflow
4. Evidence: Speed, Accuracy & Hallucinations
5. Live Demo
6. Takeaways

Reverse engineering is slow, scarce and expensive

- Malware triage and patching are **bottlenecked by human experts**: disassembly, decompilation and dynamic analysis take **hours to days** per sample.
- Volume keeps growing: **hundreds of thousands** of new samples per day, wich make a large **backlog of unpatched CVEs**.
- Skills are rare: reading x86/ARM assembly and noisy decompiler pseudo-code is hard and error-prone.

Can generative AI shrink the time-to-understanding and time-to-patch (without lying to us)?

The promise of LLMs in this workflow

LLMs can act as a **force multiplier** on top of existing tools:

- **Explain** decompiled pseudo-code in plain language.
- **Rename** functions / variables and **annotate** structures to recover intent.
- **Hypothesize** the exploited bug and **draft** a candidate mitigation or patch.

An LLM is an **accelerator and co-pilot**, **not** an autonomous replacement for the analyst. Every output must be **verified against the binary**.

The Toolchain

1. Context & Motivation
2. **The Toolchain**
3. Manual vs. AI-Augmented Workflow
4. Evidence: Speed, Accuracy & Hallucinations
5. Live Demo
6. Takeaways

The classic RE stack

Static analysis

- Disassemblers / decompilers: **Ghidra** (NSA, free), IDA Pro, Binary Ninja.
- Decompiler output = **pseudo-C** reconstructed from machine code: approximate, no original names/comments.
- `strings`, PE/ELF headers, imports.

Dynamic analysis

- Debuggers: x64dbg, GDB.
- Sandboxes: Cuckoo / CAPE, isolated VMs.
- Network / behaviour capture.

Pseudo-code ≠ source code. The decompiler guesses types and control flow, which is exactly the noisy input we ask an LLM to interpret.

The LLM-augmented stack

The LLM is **wired into** the analysis tool, not used by copy-paste:



- **MCP (Model Context Protocol)** bridges, e.g. **GhidraMCP** (LaurieWired), expose Ghidra's API (list/decompile functions, rename, cross-refs) to an LLM agent.
- **Static vs. dynamic:** bridges also exist for debuggers/sandboxes, but our focus (and most tooling) is **static decompilation & patching** (using dynamic runs to **verify**).
- The agent **iterates** (decompile → explain → rename → re-read → propose patch) with a **human in the loop**.

Manual vs. AI-Augmented Workflow

1. Context & Motivation
2. The Toolchain
3. **Manual vs. AI-Augmented Workflow**
4. Evidence: Speed, Accuracy & Hallucinations
5. Live Demo
6. Takeaways

Two lanes, same destination

Manual (traditional)

1. Load binary, map functions by hand.
2. Read assembly / pseudo-code line by line.
3. Guess intent, rename, take notes.
4. Reproduce behaviour in a sandbox.
5. Locate the bug, design a mitigation.

Hours-days, depth depends on expert skill.

AI-augmented

1. Agent decompiles (via Ghidra) & summarizes functions.
2. LLM explains + renames in bulk.
3. Analyst steers questions ("what triggers this?").
4. LLM drafts IOCs / YARA / patch.
5. **Analyst verifies** against the binary.

Minutes for triage, verification still required.

Where the AI lane helps, and where it doesn't

Dimension	Manual	AI-augmented
Speed of first-pass triage	Slow	**Fast**
Comprehension for non-experts	Low	**Much higher**
Breadth / coverage of code	Limited by time	**Broad**
Correctness guarantee	**Analyst-verified**	Needs verification
Risk of confident errors	Low	**Hallucinations**

Net effect: AI moves effort from **reading** to **verifying**.

Evidence: Speed, Accuracy & Hallucinations

1. Context & Motivation
2. The Toolchain
3. Manual vs. AI-Augmented Workflow
4. **Evidence: Speed, Accuracy & Hallucinations**
5. Live Demo
6. Takeaways

Speed & comprehension: what studies show

- **DHuman-LLM teaming study** (Eurecom, NDSS 2026, 48 participants): novices' comprehension rose by **98%**, and functions implementing **known algorithms** were triaged up to **2.4 × faster**.
- **But**: experts gained little, and some outputs contained **harmful hallucinations** and misleading suggestions.

Biggest, most reliable win: **lowering the barrier to entry** and speeding up **first-pass triage**, not replacing expert judgement.

Remediation & patching is still hard

- **PatchEval** (2025, 1,000 real CVEs, 230 in execution sandbox): the best single model fixed only **23%** (53/230). Different models solve different bugs, and combining them reaches **43%** (98/230).
- **Architecture study** (arXiv:2603.01257, 2026) compares fixed-workflow vs. single-/multi-agent vs. general code agents, trading **accuracy vs. time & token cost**.

The LLM **proposes** a patch, a human (and a test-suite) **validates** it.

The hallucination problem

Failure modes reported repeatedly in the literature:

- **Invented logic**: plausible-sounding but wrong explanations.
- **Misleading renaming**: a function named `decrypt_payload` that doesn't decrypt.
- **Low recompilability**: DecLLM (ISSTA 2025) raised recompilable decompiled code from **9% → 25%**. I.e. most output still **doesn't even rebuild**.

- **Why automated malware classification fails**: LLMs only read static text (without executing the binary), so obfuscation and hallucinated semantics lead to **wrong family/maliciousness labels**.
- **Mitigation → verify against ground truth:** recompile, diff behaviour, symbolic execution, unit tests.

Live Demo

1. Context & Motivation
2. The Toolchain
3. Manual vs. AI-Augmented Workflow
4. Evidence: Speed, Accuracy & Hallucinations
5. **Live Demo**
6. Takeaways

Demo: before / after

The "before": in 2017, stopping **WannaCry** meant an analyst reading the malware's code live during a global outbreak to find the **kill-switch domain**. Manual, under pressure, hours of work.

The "after" (our demo, stopwatch on screen):

1. **Act A - Accelerated understanding:** Ghidra + LLM (via GhidraMCP) explains a documented sample in an **isolated VM** → behaviour summary, **IOCs** and a **YARA** rule + mitigation (disable SMBv1 / apply MS17-010).
2. **Act B - End-to-end remediation:** our **own** vulnerable toy binary → LLM spots the bug in the pseudo-code, **proposes the patch**, we recompile → the crash is gone.

Live Demo

- **Play recorded demo**

Takeaways

1. Context & Motivation
2. The Toolchain
3. Manual vs. AI-Augmented Workflow
4. Evidence: Speed, Accuracy & Hallucinations
5. Live Demo
6. **Takeaways**

Takeaways

- LLMs **accelerate** reverse engineering and **draft** remediations. The biggest wins are in **triage** and for **non-experts**.
- They do **not** replace analysts: patch success is **modest** (23%) and **hallucinations** are real.
- The discipline that makes it work: **keep a human in the loop** and **verify every output against the binary**.
- Pure LLM **malware classification** remains unreliable because static models **cannot see runtime behaviour**, where obfuscation and hallucinations lead to wrong labels without execution verification.

References

- **Human-LLM teaming in RE:** *"Decompiling the Synergy: An Empirical Study of Human-LLM Teaming in Software Reverse Engineering"*, Eurecom, NDSS 2026.
- **Decompiler refinement: DeGPT** (NDSS 2024), **LLM4Decompile** (EMNLP 2024, arXiv:2403.05286), **SoK: LLMs for RE** (arXiv:2509.21821, 2025).
- **Recompilability & verification: DecLLM** (ISSTA 2025), **D-LiFT / D-Score** (arXiv:2506.10125, 2025).
- **Automated patching: PatchEval** (arXiv:2511.11019, 2025), **LLM-Based Architectures for Automated Patching** (arXiv:2603.01257, 2026).
- **Tooling & context: Ghidra** (NSA, ghidra-sre.org), **GhidraMCP** (LaurieWired, GitHub), **AV-TEST** Malware Statistics, **WannaCry** kill-switch analysis (2017).

Thank you!

Questions?